

UNIwersytet Ekonomiczny w Katowicach
Kierunek Informatyka Analiza danych

Adam Kowalczyk

Projekt 2: Systemy uczące się w analizie danych

Analiza regresji z cechami wpływającymi na cenę domu

KATOWICE 2025

Spis treści

1. Cel projektu	2
2. Zbiór danych	3
3. Wstępna analiza danych	6
4. Transformacje danych	7
5. Trenowanie modeli	11

1. Cel projektu

Celem projektu było zbudowanie i ocena modeli predykcyjnych szacujących ceny nieruchomości w Ames (stan Iowa w USA) na podstawie cech numerycznych i kategoriowych opisujących warunki zamieszkania zawartych w zbiorze danych pobranym z Kaggle- "House Prices: Advanced Regression Techniques". Przeprowadzono analizę eksploracyjną danych, ocenę rozkładów zmiennych, zidentyfikowano wartości odstających oraz porównanie jakości predykcji dwóch modeli regresyjnych.

2. Zbiór danych

Dane użyte w tej analizie zawierają wiele cech wpływających na końcową cenę nieruchomości. Zbiór ten został stworzony z myślą o trenowaniu i testowaniu zaawansowanych technik regresji- przygotowany więc jest w celach szkoleniowych prezentując ćwiczenie dla użytkowników chcących nauczyć się stosowania i udoskonalania metod regresyjnych.

W pliku "train.csv" do pobrania z tego źródła znajduje się 1460 obserwacji na podstawie 80 cech i zmiennej docelowej "SalePrice". Cechy dzielą się na 43 numeryczne i 37 kategoriowych:

- SalePrice: cena sprzedaży nieruchomości w dolarach,
- MSSubClass: klasa budynku,
- MSZoning: ogólna klasyfikacja strefy,
- LotFrontage: długość ulicy połączonej z nieruchomością,
- LotArea: wielkość działki w stopach kwadratowych,
- Street: typ dostępu do drogi,
- Alley: typ dostępu do alei,
- LotShape- ogólny kształt nieruchomości,
- LandContour- płaskość terenu nieruchomości,
- Utilities: dostęp do prądu/gazu/wody,
- LotConfig: konfiguracja działki,
- LandSlope: nachylenie terenu nieruchomości,
- Neighborhood: fizyczne lokalizacje w granicach miasta Ames,
- Condition1: bliskość mieszkania do głównej drogi lub linii kolejowej,

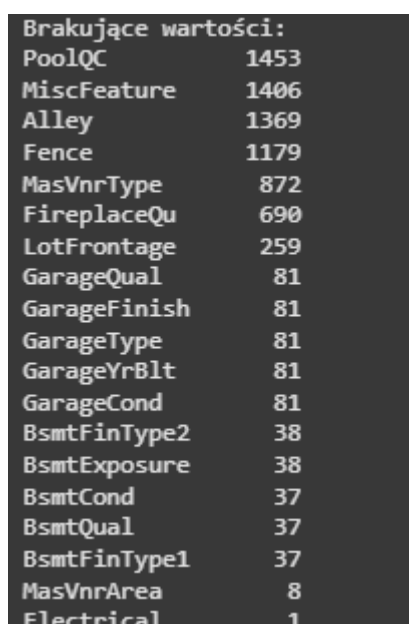
- Condition2: bliskość mieszkania do głównej drogi lub linii kolejowej (jeśli istnieją więcej niż jedna),
- BldgType: typ mieszkania (jednorodzinny wolnostojący/duplex itp.),
- HouseStyle: styl mieszkania,
- OverallQual: ogólna jakość materiałów i wykończenia (1-10),
- OverallCond: ogólna ocena stanu mieszkania (1-10),
- YearBuilt: oryginalna data budowy,
- YearRemodAdd: data przebudowy,
- RoofStyle: typ dachu,
- RoofMatl: materiał dachu,
- Exterior1st: pokrycie zewnętrzne domu (ściany),
- Exterior2nd: pokrycie zewnętrzne domu (jeśli więcej niż jeden materiał),
- MasVnrType: typ forniru murowanego,
- MasVnrArea: powierzchnia forniru murowanego w stopach kwadratowych,
- ExterQual: jakość materiału zewnętrznego,
- ExterCond: obecny stan materiału na zewnątrz,
- Foundation: typ fundamentu,
- BsmtQual: wysokość piwnicy,
- BsmtCond: ogólny stan piwnicy.
- BsmtExposure: ściany w przejściu lub na poziomie ogrodu do piwnicy,
- BsmtFinType1: jakość wykończonej powierzchni piwnicy,
- BsmtFinSF1: typ 1 powierzchni użytkowania piwnicy w stopach kwadratowych,
- BsmtFinType2: jakość wykończenia piwnicy (jeśli więcej niż 1 typ),
- BsmtFinSF2: typ 2 powierzchni użytkowania piwnicy w stopach kwadratowych,
- TotalBsmtSF: Łączna powierzchnia piwnicy w stopach kwadratowych,
- Heating: typ ogrzewania,
- HeatingQC: jakość i stan ogrzewania,
- CentralAir: centralna klimatyzacja,
- Electrical: system elektryczny,
- 1stFlrSF: powierzchnia pierwszego piętra,
- 2ndFlrSF: powierzchnia drugiego piętra,
- LowQualFinSF: Niska jakość wykończenia (wszystkie piętra)
- GrLivArea: powierzchnia mieszkalna nad poziomem gruntu w stopach kwadratowych

- BsmtFullBath: pełne łazienki w piwnicy,
- BsmtHalfBath: pół łazienki w piwnicy,
- FullBath: pełne łazienki powyżej poziomu,
- HalfBath: pół łazienki w piwnicy powyżej poziomu,
- Bedroom: liczba sypialni powyżej poziomu piwnicy,
- Kitchen: liczba kuchni,
- KitchenQual: jakość kuchni,
- TotRmsAbvGrd: całkowita liczba pokoi powyżej poziomu (nie obejmuje łazienek),
- Functional: ocena funkcjonalności domu,
- Fireplaces: liczba kominków,
- FireplaceQu: jakość kominka,
- GarageType: lokalizacja garażu,
- GarageYrBlt: rok budowy garażu,
- GarageFinish: wykończenie wnętrza garażu,
- GarageCars: rozmiar garażu do pomieszczenia samochodu,
- GarageArea: rozmiar garażu w stopach kwadratowych,
- GarageQual: jakość garażu,
- GarageCond: stan garażu,
- PavedDrive: utwardzony podjazd,
- WoodDeckSF: powierzchnia drewnianego tarasu w stopach kwadratowych,
- OpenPorchSF: powierzchnia otwartego ganku w stopach kwadratowych,
- EnclosedPorch: powierzchnia zamkniętego ganku w stopach kwadratowych,
- 3SsnPorch: powierzchnia werandy w stopach kwadratowych,
- ScreenPorch: powierzchnia ganku w stopach kwadratowych,
- PoolArea: powierzchnia basenu w stopach kwadratowych,
- PoolQC: jakość basenu,
- Fence: jakość ogrodzenia,
- MiscFeature: inne cechy (windy, pole do tenisa, drugi garaż...)
- MiscVal: wartość innych cech (z punktu powyżej) w dolarach
- MoSold: miesiąc sprzedaży,
- YrSold: rok sprzedaży
- SaleType: typ sprzedaży,
- SaleCondition: kondycja sprzedaży (normalna, wymiana, pomiędzy rodziną...)

- ID

3. Wstępna analiza danych

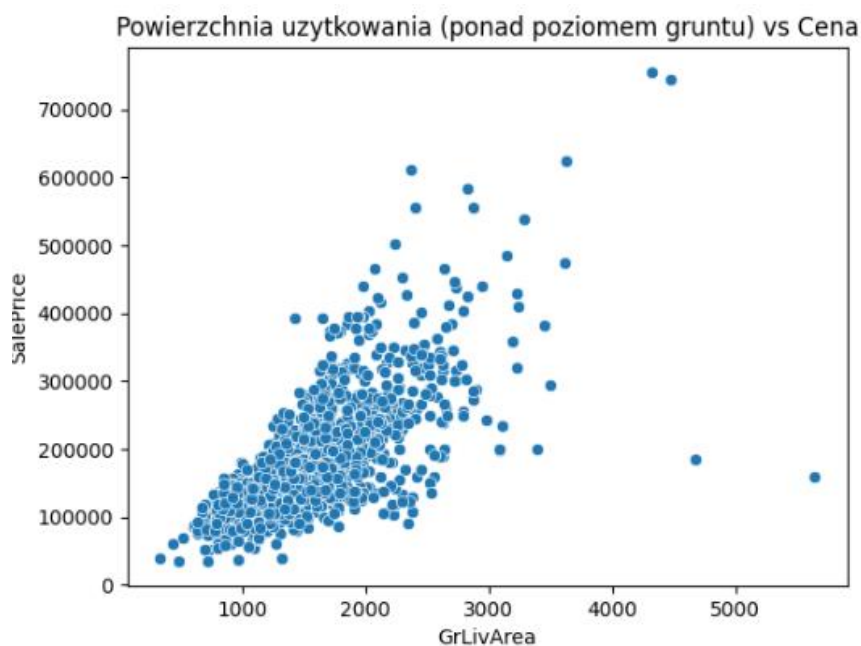
Zbiór zawiera duże ilości braków w danych. Wynika to głównie z zostawianych pustych pól w przypadku braku posiadania przez zamieszkanie danej cechy (np. basenu).



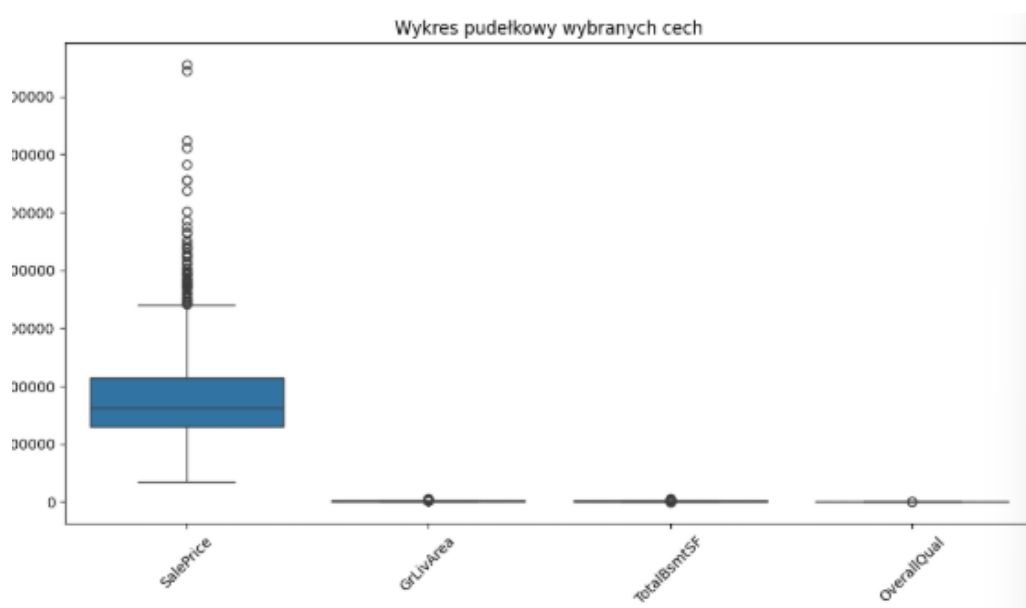
Brakujące wartości:	
PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
MasVnrType	872
FireplaceQu	690
LotFrontage	259
GarageQual	81
GarageFinish	81
GarageType	81
GarageYrBlt	81
GarageCond	81
BsmtFinType2	38
BsmtExposure	38
BsmtCond	37
BsmtQual	37
BsmtFinType1	37
MasVnrArea	8
Electrical	1

Rysunek 1- Lista cech o największej ilości pustych pól.

Zbiór nie zawiera duplikatów. Na podstawie testu Shaphiro-Wilka, D'Agostino-Pearsona i Andersona_Darlinga, wykazano, że cechy nie zawierają rozkładu normalnego (nawet "SalePrice") i nie były wcześniej zestandaryzowane. Duża ilość cech zawiera wartości odstające.

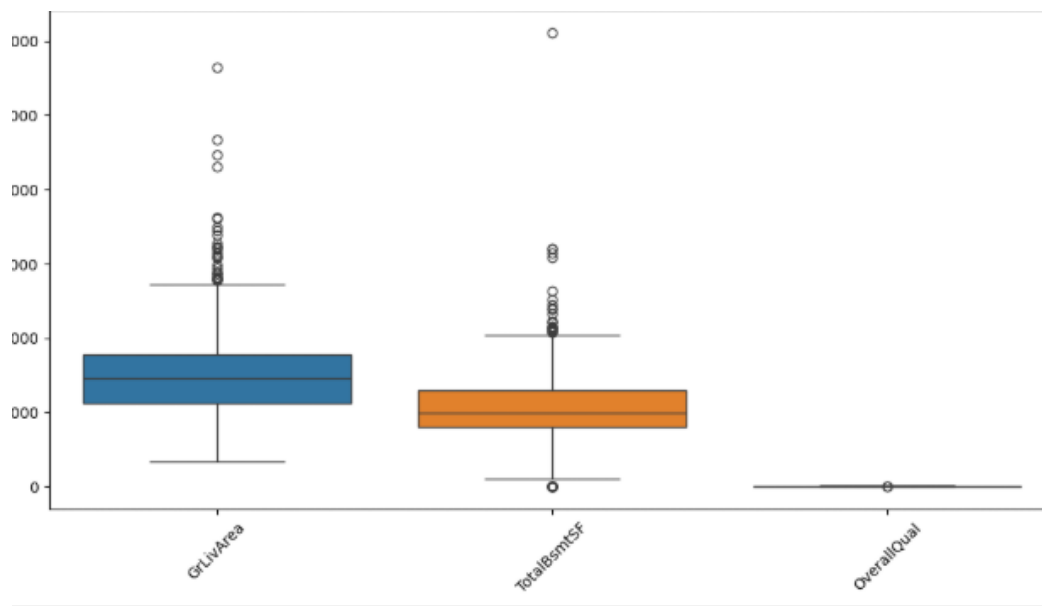


Rysunek 2- Przykład odstających wartości cechy “GrLivArea” na cenę mieszkania.



Rysunek 3- Wykres pudełkowy przedstawiający rozproszone wartości danych ceny mieszkania.

Widoczna jest asymetryczność rozkładu. Można też dość do wniosku, że zbiór danych zawiera większość mieszkań w tania a niewiele bardzo drogich. Na poniższym wykresie również można zauważyć podobne wnioski dla innych zmiennych.



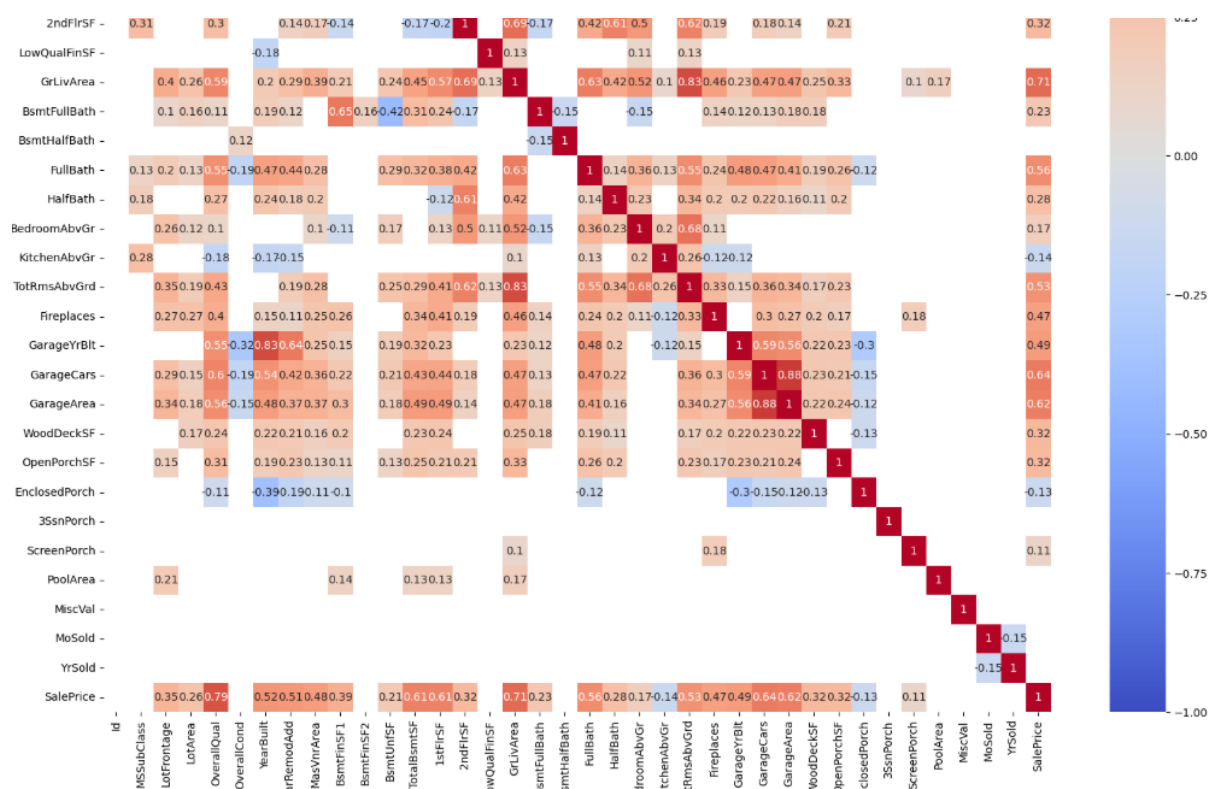
Rysunek 3- wykresy pudełkowe kilku innych cech.

Przygotowano macierz korelacji (Rysunek 4), na którym można zauważyć, że najsilniejsze korelacje ze zmienną docelową posiadają cechy:

- OverallQual: 0.79 (jakość materiałów i wykończenia)
- GrLivArea: 0.71 (powierzchnia mieszkalna)
- GarageCars: 0.64 (wielkość garażu)
- GarageArea: 0.62 (powierzchnia garażu)
- TotalBsmtSF: 0.61 (powierzchnia piwnicy)

Dodatkowo zaobserwować można silne korelacje pomiędzy innymi cechami niż zmienna docelowa:

- GarageCars i GarageArea: 0.88 (rozmiar miejsca na samochody i ogólny rozmiar garażu),
- TotalBsmtSF i 1stFlrSF: 0.82 (powierzchnia piwnicy i pierwszego piętra),
- GrLivArea i TotRmsAbvGrd: 0.83 [powierzchnia użytkowania i całkowita liczba pokoi powyżej poziomu (nie obejmuje łazienek)]



Rysunek 4- część macierzy korelacji prezentującej relacje ze zmienną docelową (puste pola oznaczają korelację wynoszącą mniej niż 0,1)

4. Transformacje danych

Aby modele regresji uzyskały jak najlepsze wyniki, wpięrw dokonano dokładnego czyszczenia danych. Pierwszym etapem tego procesu było pozbycie się wartości odstających. Na podstawie wcześniej przygotowanych histogramów i wykresów punktowych, po kolei sprawdzono dla jakich cech wartości zbyt przekraczają skalę większości wyników aby je można usunąć ze zbioru. Lista warunków dla których wartości zostały usunięte znajduje się na rysunku 5. Dzięki temu czyszczeniu, usunięte zostało 38 obserwacji.

```

#warunki do usunięcia
conditions = [
    df['GrLivArea'] > 4200,
    df['LotFrontage'] > 250,
    df['LotArea'] > 50000,
    df['OverallCond'] == 2,
    (df['OverallCond'] == 5) & (df['SalePrice'] > 700000),
    (df['OverallCond'] == 6) & (df['SalePrice'] > 700000),
    df['MasVnrArea'] > 1400,
    df['BsmtFinSF1'] > 5000,
    df['BsmtFinSF2'] > 1200,
    (df['BsmtFinSF2'] > 400) & (df['SalePrice'] > 500000),
    df['TotalBsmtSF'] > 5000,
    df['LowQualFinSF'] > 530,
    df['BsmtFullBath'] == 3,
    df['BsmtHalfBath'] == 2,
    df['BedroomAbvGr'] > 7,
    df['KitchenAbvGr'] == 3,
    df['TotRmsAbvGrd'] > 13,
    df['GarageArea'] > 1200,
    df['WoodDeckSF'] > 800,
    df['OpenPorchSF'] > 500,
    df['EnclosedPorch'] > 500
]

```

Rysunek 5- Przygotowanie do usunięcia wartości odstających z wybranych cech.

Następnie zajęto się obsługą brakujących danych w zbiorze. Na podstawie wcześniej przedstawionej (rysunek 1) liście cech o największej ilości braków w danych, usunięto 6 cech z największą ilością pustych wartości. Usunięto również “GarageArea” z uwagi na jej wysoką korelację z “GarageCars” i bardzo podobny wpływ na zmienną docelową.

```

#2. Obsługa brakujących danych
def delete_missing_data(df):
    cols_to_drop = ['PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu', 'MasVnrType', 'GarageArea']
    df = df.drop(columns=cols_to_drop, errors='ignore')
    return df

train_processed = delete_missing_data(train_clean)

```

Rysunek 6- usunięcie wybranych cech.

W kolejnym etapie zajęto się uzupełnieniem pustych wartości w kolejnych cechach, które miały większy wpływ na cenę i nie zawierały wielkiej ilości braków. Jak wcześniej wspomniano, niewypełnione dane były wpływem nieuzupełniania wartości, gdy mieszkanie nie posiadało wybranej cechy. Dlatego takie braki uzupełniono w wartościach kategoriycznych poprzez dodanie wartości “No” a w wartościach liczbowych poprzez dodanie “0”. W

przypadku pozostałych kolumn, wartości puste w przypadku danych liczbowych zastąpiono medianą w danej kolumnie. W przypadku kolumn kategorycznych, gdzie braki były spowodowane innym powodem niż brak cechy, puste wartości zostały zastąpione przez wartości najczęściej występujące w danej kategorii. Zmiana ta minimalizuje zniekształcenie danych. Warto zaznaczyć, że bardzo niewiele cech zostało przekształcone w ten sposób, gdyż większość pozostałych braków została już naprawiona przez poprzednie metody.

```
#3. Edycja cech z mniejszą ilością brakujących danych
def handle_missing_data(df):

    df['MasVnrArea'] = df['MasVnrArea'].fillna(0)
    df['GarageType'] = df['GarageType'].fillna('No')
    df['GarageFinish'] = df['GarageFinish'].fillna('No')
    df['GarageQual'] = df['GarageQual'].fillna('No')
    df['BsmtQual'] = df['BsmtQual'].fillna('No')
    df['BsmtCond'] = df['BsmtCond'].fillna('No')
    df['BsmtExposure'] = df['BsmtExposure'].fillna('No')
    df['BsmtFinType1'] = df['BsmtFinType1'].fillna('Unf')
    df['Electrical'] = df['Electrical'].fillna('SBrkr')
    df['LotFrontage'] = df['LotFrontage'].fillna(0)

    #Strategie dla pozostałych kolumn
    #Numeryczne - mediana
    num_cols = df.select_dtypes(include=[np.number]).columns
    for col in num_cols:
        if df[col].isnull().sum() > 0 and col not in ['MasVnrArea', 'LotFrontage']:
            df[col] = df[col].fillna(df[col].median())

    #Kategoryczne - najczęstsza wartość
    cat_cols = df.select_dtypes(include=['object']).columns
    for col in cat_cols:
        if df[col].isnull().sum() > 0 and col not in ['GarageType', 'GarageFinish', 'GarageQual']:
            df[col] = df[col].fillna(df[col].mode()[0])

    return df
```

Rysunek 7- usunięcie braków dla kolumn z mniejszą ilością braków.

Aby usunąć prawostronność skośną w rozkładzie zmiennej docelowej “SalePrice”, poddano ją logarytmowaniu. Dzięki temu rozkład jest bardziej symetryczny i zredukowany jest wpływ odstających wartości. Ułatwia to również interpretację- zmiany stają się procentowe. Przekształconą zmienną nazwano “SalePrice_log”.

Kolejnym krokiem było konieczne zamienienie wartości kategorycznych na liczbowe. Dzięki temu algorytmy uczenia będą operować na jednym typie

danych. Każda kategoria zostaje przekształcona w nową kolumnę binarną (0 lub 1) na zasadzie “One-Hot encoding”. Domyślnie pierwsza kategoria jest pomijana aby uniknąć redundancji. Wybierane są kolumny katagoryczne z przynajmniej dwoma unikalnymi wartościami. Ten sposób nie narzuca sztucznej kolejności poprzez sztuczne modyfikacje i dobrze sobie radzi z wcześniej poprawionymi pustymi wartościami. Duża ilość kolumn generowanych w ten sposób nie wpłynie znacząco na dwie wybrane później modele regresji, bo te dobrze radzą sobie z dużą ilością cech.

Dane liczbowe są poddawane standaryzacji. Zastosowano to z powodu różnych typów danych liczbowych różnych cechach (ceny, rozmiar, skala...). W wyniku tej operacji skala cen została ujednolicona co poprawiło wydajność i interpretowalność przyszłych modeli. W metodzie “standarize_features” wpierv zachowano zmienną docelową i “ID” aby dodać je w późniejszym etapie.

```
train_processed['SalePrice_log'] = np.log1p(train_processed['SalePrice'])

#4. Przekształcenie zmiennych katagorycznych na liczbowe (One-Hot encoding)
def encode_categorical(df):
    cat_cols = [col for col in df.select_dtypes(include=['object']).columns
                 if df[col].nunique() > 1]

    df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)
    return df_encoded

train_encoded = encode_categorical(train_processed)

#5. Standaryzacja cech numerycznych
def standarize_features(df, target_col='SalePrice_log'):
    if target_col in df.columns:
        target = df[target_col].copy() #kopia wartości docelowej i ID aby je zachować
        df = df.drop(columns=[target_col, 'SalePrice'], errors='ignore')
    else:
        target = None

    numeric_cols = df.select_dtypes(include=[np.number]).columns

    scaler = StandardScaler()
    df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

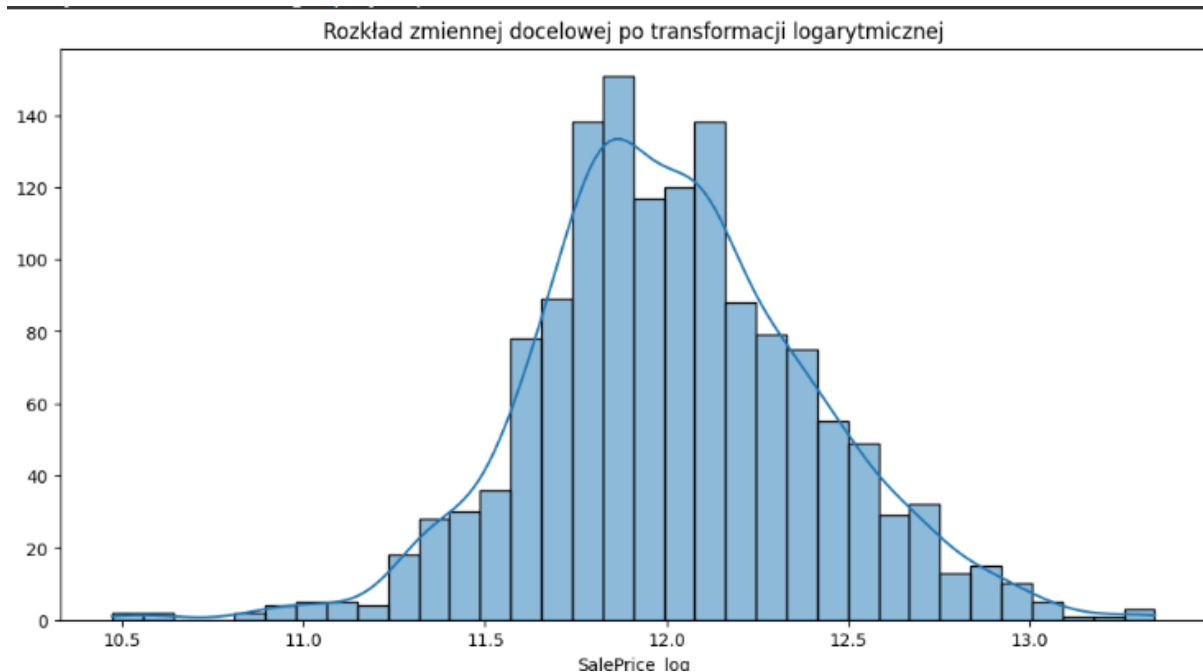
    #przywracanie zmiennej docelowej
    if target is not None:
        df[target_col] = target

    return df, scaler
```

Rysunek 8- logarytmowanie “SalePrice”, zamiana danych katagorycznych na liczbowe, standaryzacja.

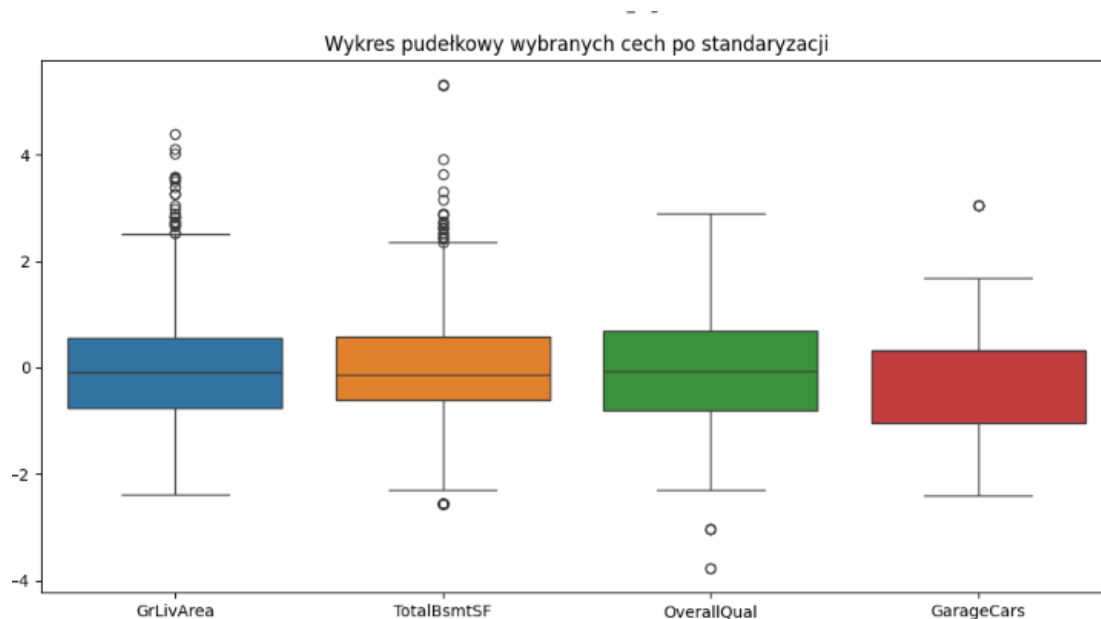
Po przeprowadzeniu tych operacji, usunięto ze zbioru zmienną docelową i niepotrzebną cechę ”ID”. Dane podzielono na zbiór uczący i testowy. Aby

sprawdzić rozkład zmiennej docelowej po jej transformacji, przygotowano wykres zaprezentowany na rysunku 9. Jak można zauważyć, prawostronność skośna została skutecznie bardzo zmniejszona i cały histogram przypomina rozkład normalny. Wpływ wartości odstających został istotnie zredukowany.



Rysunek 9- Rozkład "SalePrice_log"

Wykresy pudełkowe czterech wybranych cech po standaryzacji potwierdzają, że proces standaryzacji przebiegł pomyślnie. Mało wartości w "OverallQual" i "GarageCars" jest odstajaca. "GrLivArea" i "TotalBsmtSF" są bardziej zróżnicowane.



Rysunek 10- wykres pudełkowy dla czterech cech po standaryzacji.

5. Trenowanie modeli

Po dokonanych zmianach, zbiór danych nadaje się do treningu. Jako modele regresji wybrano “Random Forest Regressor” oraz “Gradient Boosting Regressor”. Modele ocenione zostaną za pomocą czterech metryk:

- MAE (średni błąd bezwzględny),
- MSE (średni błąd kwadratowy),
- RMSE (pierwiastek z MSE),
- R^2 (współczynnik determinacji).

Dla modelu “Random Forest” wprowadzono następujące ustawienia:

- random_state = 42 (ziarno losowości),
- n_estimators = 100 (liczba drzew),
- max_depth= 4 (ograniczenie maksymalnej głębokości drzewa, zapobiega przeuczeniu),

- `min_samples_leaf= 5` (minimalna liczba próbek w liściach, ograniczenie małych przeuczonych gałęzi),
- `min_samples_split= 10` (liczba próbek wymagana do podziału węzła),
- `max_features= 'sqrt'` (liczba cech branych pod uwagę przy podziale)

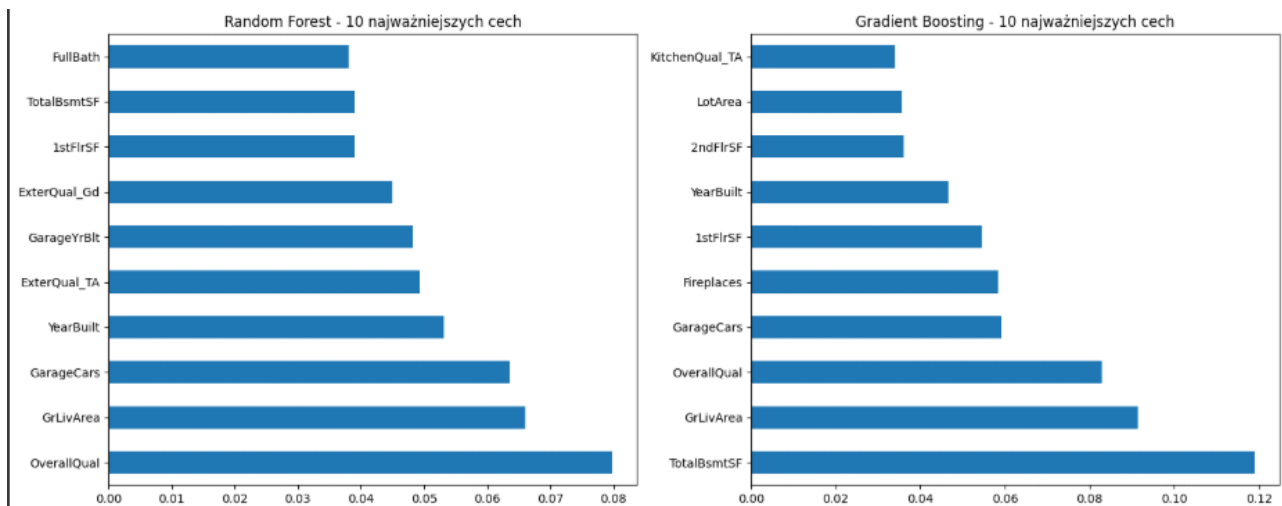
Dla modelu “Gradient Boosting” wprowadzono następujące ustawienia:

- `random_state= 42`,
- `n_estimators= 100`,
- `learning_rate= 0.05` (współczynnik uczenia),
- `max_depth= 2`,
- `subsample= 0.8` (procentowa wartość próbek używanych do trenowania każdego drzewa),
- `min_samples_leaf= 5`,
- `min_samples_split= 10`,
- `max_features= 'sqrt'`

	Random Forest	Gradient Boosting
Train MSE	0.0319	0.0180
Test MSE	0.0405	0.0234
Train RMSE	0.1787	0.1343
Test RMSE	0.2011	0.1529
Train MAE	0.1258	0.0954
Test MAE	0.1370	0.1017
Train R2	0.7828	0.8773
Test R2	0.7478	0.8543

Rysunek 11- Wyniki pierwszego uczenia modeli regresji.

Rysunek 11 przedstawia wyniki dla modeli o podanych parametrach. Jak można zauważyć, błędy MAE i MSE nie osiągają wysokich wartości dla obu modeli a współczynnik R^2 wynosi wartości zbliżone do 1. Zatem oba modele tłumaczą dużą część zmienności. Różnice w wartościach treningowych i testowych również nie posiadają znacznych różnic. Nie obserwuje się zatem wysokiego przeuczenia. Ostatecznie “Gradient Boosting” osiąga lepsze wyniki pod względem wszystkich kluczowych metryk regresji zarówno na zbiorze treningowym jak i testowym. Rysunek 12 przedstawia, które cechy były najważniejsze dla danych modeli.



Rysunek 12- 10 najważniejszych cech dla "Random Forest" i "Gradient Boostng".

Pomimo zadowalających wyników, podjęto próbę poprawienia modeli poprzez wyszukanie najlepszych parametrów dla "Random Forest" i "Gradient Boosting" dzięki "GridSearchCV" z pięciokrotną walidacją krzyżową (cv= 5).

```
#Szukanie najlepszych parametrów dla random Forest i Gradient Boosting
rf_params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

rf_grid = GridSearchCV(RandomForestRegressor(random_state=42), param_grid=rf_params, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
rf_grid.fit(X_train, y_train)

gb_params = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth': [3, 4, 5]
}

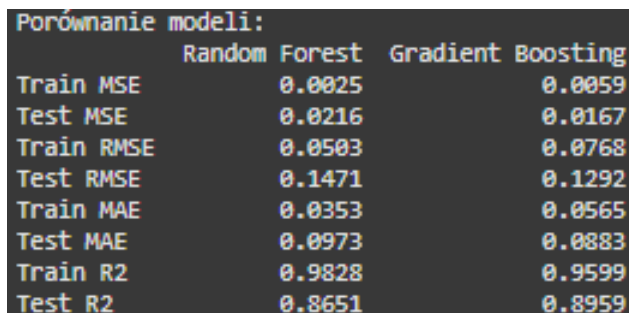
gb_grid = GridSearchCV(GradientBoostingRegressor(random_state=42), param_grid=gb_params, cv=5, scoring='neg_mean_squared_error')
gb_grid.fit(X_train, y_train)

print("Najlepsze parametry Random Forest:", rf_grid.best_params_)
print("Najlepsze parametry Gradient Boosting:", gb_grid.best_params_)
```

Rysunek 13- Szukanie najlepszych parametrów dla metod regresji.

Po zakończeniu procesu wyszukiwania, dla "Random Forest" otrzymano "max_depth" = 15, "min_samples_split" = 2 i "n_estimators" = 200. Algorytm zatem zaproponował wyższe wartości parametrów niż założono osobiście w pierwszej próbie. Dla modelu "Gradient Boosting" propozycje "learning_rate"= 0,05, "max_depth"= 3 i "n_estimators"= 200 również wskazały podobną

tendencję rekomendacji z zachowaniem współczynnika uczenia na obecnym poziomie.



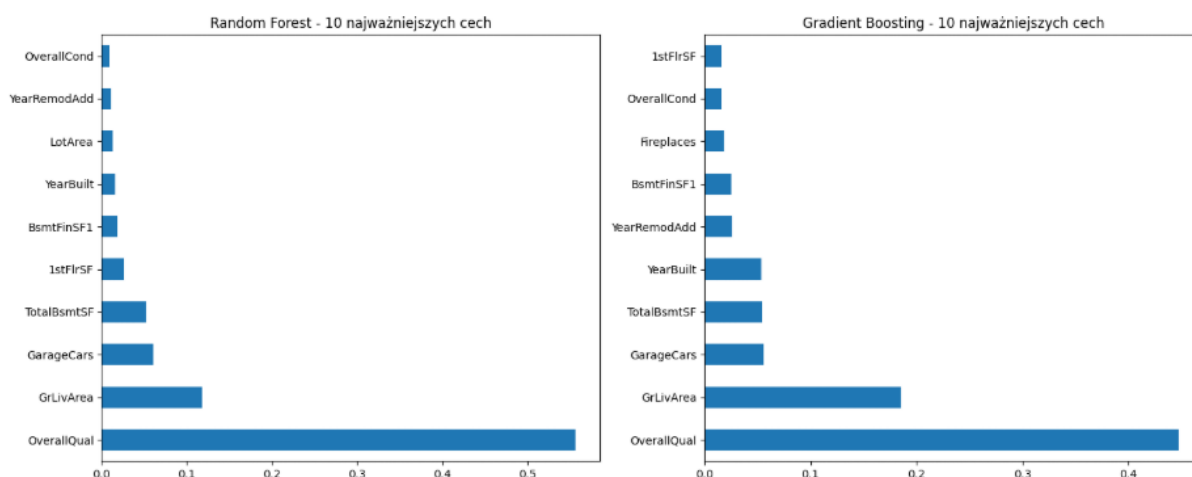
Porównanie modeli:

	Random Forest	Gradient Boosting
Train MSE	0.0025	0.0059
Test MSE	0.0216	0.0167
Train RMSE	0.0503	0.0768
Test RMSE	0.1471	0.1292
Train MAE	0.0353	0.0565
Test MAE	0.0973	0.0883
Train R2	0.9828	0.9599
Test R2	0.8651	0.8959

Rysunek 14- Wyniki uczenia modeli przy dobraniu lepszych parametrów.

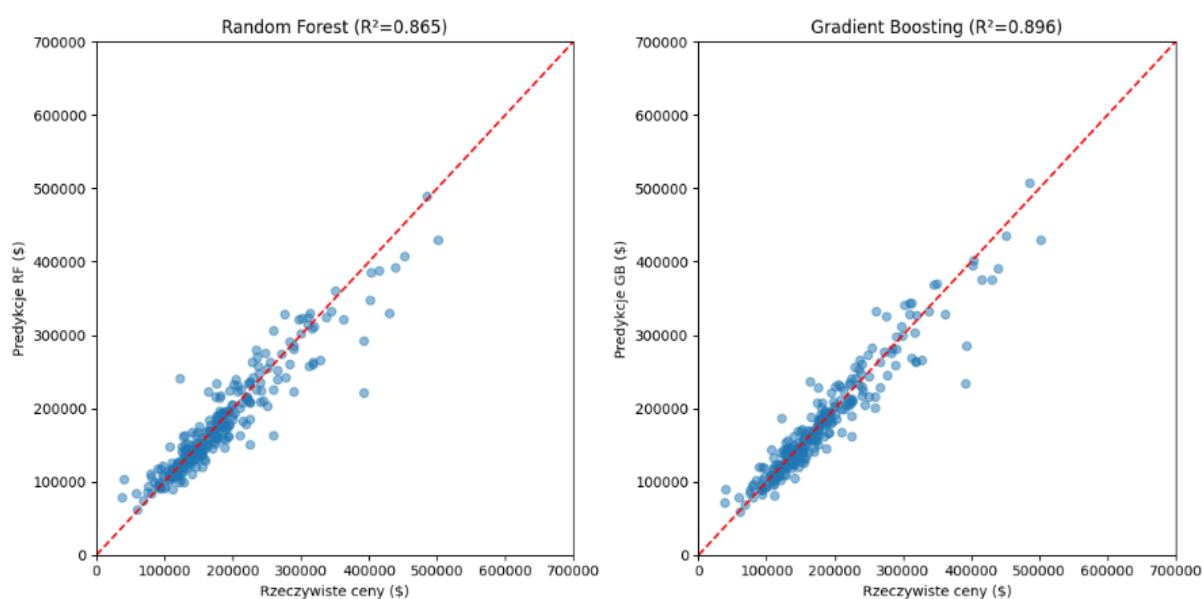
Na podstawie wyników analizy (rysunek 14), widzimy poprawę we wszystkich metrykach. Błędy MSE i MAE zmniejszyły się we wszystkich przypadkach a współczynnik R^2 osiągnął wartości jeszcze bardziej bliższe 1 niż poprzednio. Widzimy jednak, że “Random Forest” wykazuje większe przeuczenie z powodu większych różnicy wartości pomiędzy zbiorem treningowym i testowym. “Gradient Boosting” osiąga nieco wyższe błędy treningowe, ale zawiera mniejsze różnice pomiędzy zbiorami treningowymi i testowymi, co świadczy o lepszej generalizacji. Podsumowując, oba modele zostały bardzo dobrze dopasowane do danego problemu, lecz “Gradient Boosting” okazał się lepszym modelem do wykorzystania w predykcji cen domów.

Rysunek 15 pokazuje, że najważniejszymi cechami w trakcie tej próby była “OverallQual” i “GrLivArea” co jest sensowne z uwagi na wyznaczoną wcześniej wysoką korelację tych zmiennych ze zmienną docelową. Są to również najbardziej istotne cechy w realnej sprzedaży domów.



Rysunek 15- Najważniejsze cechy dla modeli po dobraniu lepszych parametrów.

Ostatni rysunek 16 prezentuje wykres predykcji uczonych modeli na faktyczną cenę nieruchomości. Jak można zauważyć, występuje skumulowanie się punktów przy czerwonej linii co oznacza dobre dopasowanie. Z uwagi na bardzo dobre obecne wyniki, na tym etapie zakończono uczenie modeli.



Rysunek 16- wykres predykcji cen mieszkań dla uczonych modeli.

